

DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY:	
NAME	B. Pooja Sree
ROLL NO	AV.SC.U4CSE25103
YEAR/SEM/SECTION	2ND /3RD/CSE 'B'
CREDITS	05

SUBMITTED TO:	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR (SI.Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	PROGRAM	PAGE NO
1	WEEK1: 1(A). A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". 1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power) where: • p = principal growth factor • n = number of years 1(C). An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. 1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. 1(E). Write a recursive Python program to print the first n terms of Fibonacci series.	1-9
2	WEEK2: 2(A). Write a Python program to implement linear search. 2(B) i. Write a Python program to implement binary search with sorted array as an input. 2(B) ii. Write a Python program to implement binary search without sorted array as an input	9-15
3	WEEK 3: 3(A).Write a python program to implement bubble sort where the input elements are taken from the user 3(B). Write a python program to implement insertion sort where the input elements are taken from the user 3(C). Write a python program to implement selection sort where the input elements are taken from the user	16-21

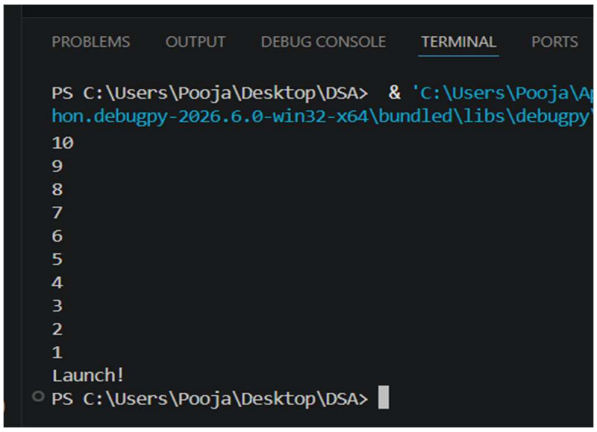
WEEK-1

1(A).AIM: A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by “Launch!”.

PROGRAM:

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)
n = int(input("Enter countdown number: "))
countdown(n)
```

OUTPUT:



EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 4 spaces for each level of indentation.

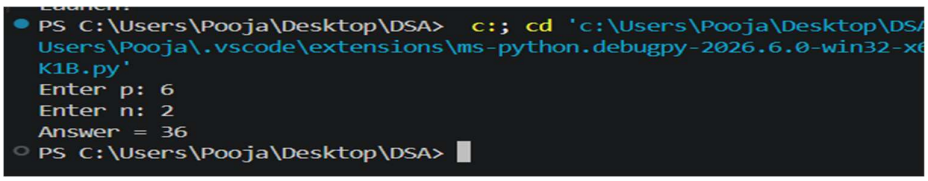
1(B).AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power) where:

- p = principal growth factor
- n = number of years

PROGRAM:

```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)
p = int(input("Enter p: "))
n = int(input("Enter n: "))
result = power(p, n)
print("Answer =", result)
```

OUTPUT:



EXPLANATION:

- The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float(input()) or int(input()).
2.	Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1).	Change the recursive call to power(p, n - 1) so that n decreases toward the base case.

1(C).AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(arr, key, index):
    if index == len(arr):
        return False
  if arr[index] == key:
        return True
 return search(arr, key, index + 1)
ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID to search: "))
if search(ids, key, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

OUTPUT:



EXPLANATION:

- The recursive function checks each employee ID in the list one by one, starting from the first element.
- If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
- If the entire list is searched without finding the ID, the function returns False.

ERROR TABLE:

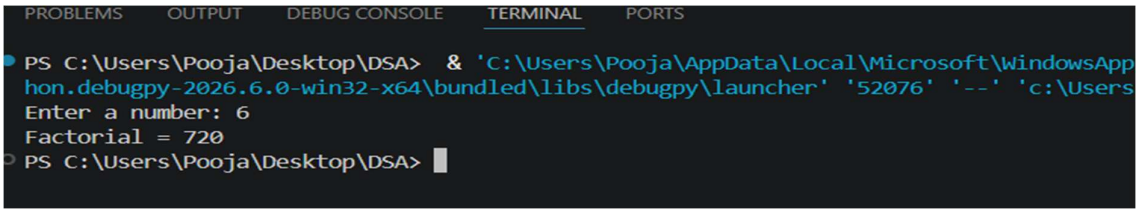
S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., employee instead of employees).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

1(D).AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
n = int(input("Enter a number: "))
print("Factorial =", factorial(n))
```

OUTPUT:



EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorial is defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2.	TypeError if a decimal number is entered for factorial calculation.	Enter only a whole number or convert the input using int().

1(E).AIM: Write a recursive Python program to print the first n terms of Fibonacci series.

PROGRAM:

```
def fibonacci(a, b, n):
    if n == 0:
        return
    print(a, end=" ")
    fibonacci(b, a + b, n - 1)
n = int(input("Enter number of terms: "))
fibonacci(0, 1, n)
```

OUTPUT:



EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

WEEK-2

2(A). AIM:Write a Python program to implement linear search.

PROGRAM:

```
arr = [50, 20, 70, 10, 40, 30]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter element to search: "))
low = 0
high = len(arr) - 1
found = False
while low <= high:
    mid = (low + high) // 2
 if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break
  elif key < arr[mid]:
        high = mid - 1
 else:
        low = mid + 1
if found == False:
    print("Element not found")
```

OUTPUT:

```
S C:\Users\Pooja\Desktop\DSA>  c:; cd 'c:\Users\Pooja\Desktop\DSA'
sers\Pooja\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64
2B(i).py'
lement found at position 4
```

EXPLANATION:

- The linear_search() function checks each element of the list one by one and compares it with the given key.
- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2(B) i.AIM: Write a Python program to implement binary search with sorted array as an input.

PROGRAM:

```
arr = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter element to search: "))
low = 0
high = len(arr) - 1
found = False
while low <= high:
    mid = (low + high) // 2
   if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break
  elif key < arr[mid]:
        high = mid - 1
 else:
        low = mid + 1
if found == False:
    print("Element not found")
```

OUTPUT:

```
                              ^C
PS C:\Users\Pooja\Desktop\DSA>  c:; cd 'c:\Users\Pooja\Desktop\DSA'; & 'C:\Users\Pooja\AppData\Local\
Users\Pooja\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '4
K2B(i).py'
Enter sorted array: 10 20 30 40 50 60
Enter element to search: 30
Element found at position 3
PS C:\Users\Pooja\Desktop\DSA>
```

EXPLANATION:

1. The program takes a sorted array as input and searches for the required element using binary search.

2. It checks the middle element and eliminates either the left or right half based on the comparison.
3. If the element is found, its position is displayed; otherwise, it prints “Element not found.”

ERROR TABLE:

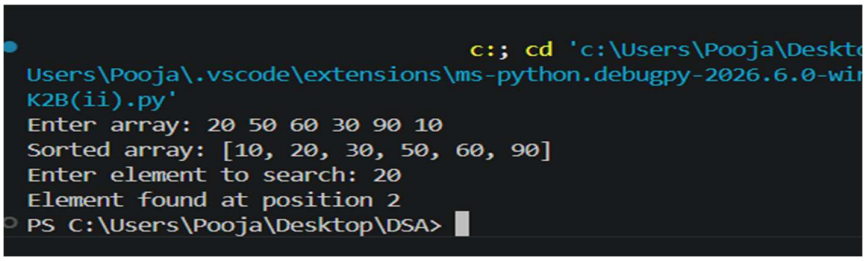
S.no.	Errors	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

2(B) ii.AIM: Write a Python program to implement binary search without sorted array as an input

PROGRAM:

```
arr = [50, 20, 70, 10, 40, 30]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter element to search: "))
low = 0
high = len(arr) - 1
found = False
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break
   elif key < arr[mid]:
        high = mid - 1
  else:
        low = mid + 1
if found == False:
    print("Element not found")
```

OUTPUT:



EXPLANATION:

1. The program takes an unsorted array as input and first sorts it using arr.sort().
2. It then uses binary search by checking the middle element and reducing the search range.
3. If the element is found, its position is displayed; otherwise, it prints “Element not found.”

ERROR TABLE:

S.no.	Errors	Rectification
1.	Logical Error	Use a found variable and print “Element not found” if it remains False.
2.	Logical Error	Use mid = (low + high) // 2 to calculate the middle index.

WEEK-3

3(A).AIM:Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
n = int(input("Enter no. of elements: "))
arr = []
print("Enter elements:")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("Sorted Array:")
for element in arr:
    print(element, end=" ")
```

OUTPUT:

```
PS C:\Users\Pooja\Desktop\DSA>  c:; cd 'c:\Users\Pooja\Desktop
Users\Pooja\.vscode\extensions\ms-python.debugpy-2026.6.0-win3
K3A.py'
Enter no. of elements: 5
Enter elements:
11
14
67
9
87
Sorted Array:
9 11 14 67 87
PS C:\Users\Pooja\Desktop\DSA>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.
2. Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.
3. After all passes, the array is sorted in ascending order and displayed.

ERROR TABLE:

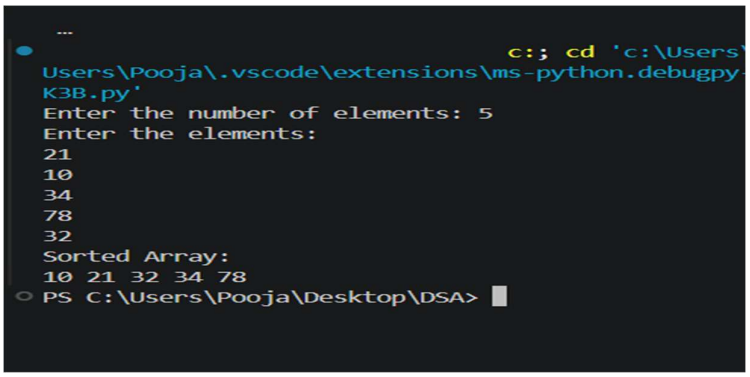
S.no.	Errors	Rectification
1.	NameError	Initialize arr = [] before taking input.
2.	Value Error	Enter only integer values

3(B).AIM:Write a python program to implement insertion sort where the input elements are taken from the user

PROGRAM:

```
def insertion_sort(arr):
    n = len(arr)
 for i in range(1, n):
        key = arr[i]
        j = i - 1
     while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j – 1
       arr[j + 1] = key
 return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted Array:")
for element in arr:
    print(element, end=" ")
```

OUTPUT:



EXPLANATION:

1.The program takes the number of elements and their values from the user and stores them in an array.

2.Insertion Sort selects each element and inserts it into its correct position among the previously sorted elements.

3. Finally, the sorted array is displayed in ascending order.

ERROR TABLE:

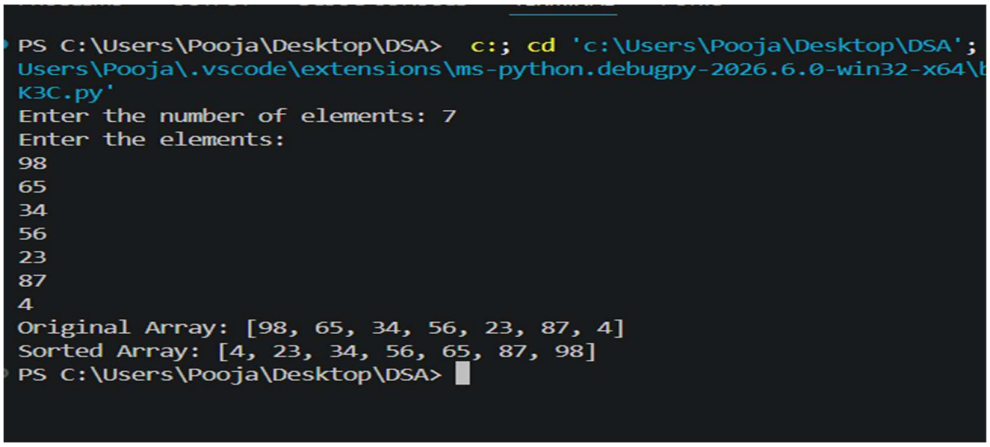
S.no.	Errors	Rectification
1.	SyntaxError	Use insertion_sort with an underscore.
2.	Name Error	Call insertion_sort(arr) correctly.

3(C).AIM:Write a python program to implement selection sort where the input elements are taken from the user

PROGRAM:

```
def selection_sort(arr):
    n = len(arr)
   for i in range(n - 1):
        min_index = I
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
  return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
print("Original Array:", arr)
selection_sort(arr)
print("Sorted Array:", arr)
```

OUTPUT:



EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.
2. Selection Sort finds the smallest element and swaps it with the first unsorted element.
3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.no.	Errors	Rectification
1.	TypeError	Convert input values using int(input())..
2.	Logic Error	Update min_index correctly and swap arr[i] with arr[min_index].